

UNIT 1: INTRODUCTION TO JAVA PROGRAMMING LANGUAGE

1.1 Properties of Java

1.2 Comparison of java with C++

1.3 Java Compiler, Java Interpreter

1.4 Identifier, Literals, Operators, Variables, Keywords, Data Types

1.5 Branching: If Else, Switch

1.6 Looping: While, Do-while, For

1.7 Type Casting

Introduction to Java Programming Language

Java is a versatile, high-level, object-oriented programming language designed for flexibility, portability, and security. It was developed by James Gosling and his team at Sun Microsystems in 1995. Today, it is owned by Oracle Corporation and continues to be one of the most popular programming languages worldwide.

It is used for:

- Mobile applications (specially Android apps)
- Desktop applications
- Web applications
- Web servers and application servers
- Database connection

Importance of Java Programming Language:

- **Widely Used:** Java powers a range of applications, from mobile apps (Android), enterprise solutions, and web applications to game development and IoT devices.
- **Rich API:** Offers a comprehensive set of libraries for almost every task, including data structures, networking, file I/O, and more.
- **Large Community:** With a vast global community, Java developers have access to extensive resources, forums, and libraries.
- **Scalability:** Ideal for both small applications and large-scale enterprise systems.
- **Backward Compatibility:** Programs written in older versions of Java often run on newer versions without modification.

Java Development Environment

To write and execute Java programs, the following components are essential:

1. **Java Development Kit (JDK):** Provides tools for developing and running Java applications.
 - The **Java Development Kit (JDK)** is a comprehensive software development environment required for building and running Java applications. It provides the essential tools and libraries for Java development and is the starting point for programmers working with Java.
 - The **JDK** is a software package that includes tools for:
 - Writing Java code.
 - Compiling the code into bytecode.
 - Debugging and running Java applications.
 - It is the primary toolkit for Java developers and serves as the foundation of the Java platform.
2. **Java Virtual Machine (JVM):** Converts Java bytecode into machine code and ensures platform independence.
 - The **Java Virtual Machine (JVM)** is the engine that enables Java's platform independence and forms the core of the Java Runtime Environment (JRE). It is responsible for executing Java programs by converting platform-independent bytecode into machine-specific instructions.
 - The **JVM** is a virtual machine that runs Java bytecode on any underlying hardware and operating system. It abstracts the details of the platform, allowing Java programs to follow the principle of "**Write Once, Run Anywhere**" (**WORA**).
3. **Java Runtime Environment (JRE):** Includes the JVM and libraries required for running Java applications.
 - The **Java Runtime Environment (JRE)** is a software package that provides the necessary components to run Java applications. It acts as an interface between Java programs and the operating system, ensuring that applications execute smoothly and efficiently.
 - The JRE is a subset of the **Java Development Kit (JDK)** designed specifically for running Java applications. It includes the Java Virtual Machine (JVM), core libraries, and other essential files required to execute Java programs.

Aspect	JDK (Java Development Kit)	JRE (Java Runtime Environment)	JVM (Java Virtual Machine)
Purpose	Provides tools for developing, compiling, and running Java applications.	Provides the environment required to run Java applications.	Executes Java bytecode and provides platform independence.
Components	Includes JRE, Java compiler, debuggers, and development tools.	Includes JVM, class libraries, and support files.	Part of the JRE; contains class loader, execution engine, and garbage collector.
For Developers/Users	Used by developers for building Java applications.	Used by end users to run Java applications.	Used internally to execute and manage Java applications.
Use Case	Writing, debugging, and compiling Java applications.	Running already compiled Java applications.	Executing Java bytecode on different platforms.
Example Command	<code>javac MyProgram.java</code>	<code>java MyProgram</code>	Managed internally by the <code>java</code> command.

1.1 Properties / Features of Java Programming Language

Following the Properties / features of java:

1. Simple

Java is very easy to learn, and its syntax is simple, clean and easy to understand. According to Sun Microsystems, Java language is a simple programming language because:

- Java syntax is based on C++ (so easier for programmers to learn it after C++).
- Java has removed many complicated and rarely-used features, for example, explicit pointers, operator overloading, etc.
- There is no need to remove unreferenced objects because there is an Automatic Garbage Collection in Java.

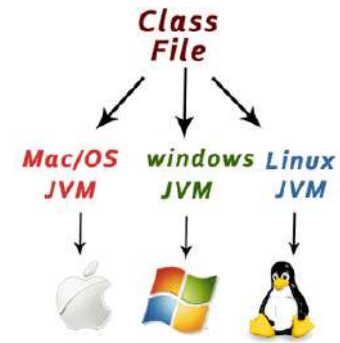
2. Object-oriented

Java is an object-oriented programming language. Everything in Java is an object. Object-oriented means we organize our software as a combination of different types of objects that incorporate both data and behavior. Object-oriented programming (OOPs) is a methodology that simplifies software development and maintenance by providing some rules. Basic concepts of OOPs are:

- Object
- Class
- Inheritance
- Polymorphism
- Abstraction
- Encapsulation

3. Platform Independent

Java is platform independent because it is different from other languages like C, C++, etc. which are compiled into platform specific machines while Java is a write once, run anywhere language. A platform is the hardware or software environment in which a program runs. There are two types of platforms software-based and hardware-based. Java provides a software-based platform. The Java platform differs from most other platforms in the sense that it is a software-based platform that runs on top of other hardware-based platforms. It has two components:



1. Runtime Environment
2. API(Application Programming Interface)

Java code can be executed on multiple platforms, for example, Windows, Linux, Sun Solaris, Mac/OS, etc. Java code is compiled by the compiler and converted into bytecode. This bytecode is a platform-independent code because it can be run on multiple platforms, i.e., **Write Once and Run Anywhere (WORA)**.

4. Secured

Java is best known for its security. With Java, we can develop virus-free systems. Java is secured because:

- **No explicit pointer**
- **Java Programs run inside a virtual machine sandbox**
- **ClassLoader:** Classloader in Java is a part of the Java Runtime Environment (JRE) which is used to load Java classes into the Java Virtual Machine dynamically. It adds security by separating the package for the classes of the local file system from those that are imported from network sources.
- **Bytecode Verifier:** It checks the code fragments for illegal code that can violate access rights to objects.
- **Security Manager:** It determines what resources a class can access such as reading and writing to the local disk.

Java language provides these securities by default. Some security can also be provided by an application developer explicitly through SSL, JAAS, Cryptography, etc.

5. Robust

Java is robust because:

- It uses strong memory management.
- There is a lack of pointers that avoids security problems.
- Java provides automatic garbage collection which runs on the Java Virtual Machine to get rid of objects which are not being used by a Java application anymore.
- There are exception handling and the type checking mechanism in Java. All these points make Java robust.

6. Architecture-neutral

Java is architecture neutral because there are no implementation dependent features, for example, the size of primitive types is fixed. In C programming, int data type occupies 2 bytes of memory for 32-bit architecture and 4 bytes of memory for 64-bit architecture. However, it occupies 4 bytes of memory for both 32 and 64-bit architectures in Java.

7. Portable

Java is portable because it facilitates you to carry the Java bytecode to any platform. It doesn't require any implementation.

8. High-performance

Java is faster than other traditional interpreted programming languages because Java bytecode is "close" to native code. It is still a little bit slower than a compiled language (e.g., C++). Java is an interpreted language that is why it is slower than compiled languages, e.g., C, C++, etc.

9. Distributed

Java is distributed because it facilitates users to create distributed applications in Java. RMI and EJB are used for creating distributed applications. This feature of Java makes us able to access files by calling the methods from any machine on the internet.

10. Multi-threaded

A thread is like a separate program, executing concurrently. We can write Java programs that deal with many tasks at once by defining multiple threads. The main advantage of multi-threading is that it doesn't occupy memory for each thread. It shares a common memory area. Threads are important for multi-media, Web applications, etc.

11. Dynamic

Java is a dynamic language. It supports the dynamic loading of classes. It means classes are loaded on demand. It also supports functions from its native languages, i.e., C and C++. Java supports dynamic compilation and automatic memory management (garbage collection).

1.2 Comparison of JAVA with C++

Sno.	Comparison	C++	Java
1	Platform-independent	C++ is platform-dependent.	Java is platform-independent.
2	Mainly used for	C++ is mainly used for system programming.	Java is mainly used for application programming. It is widely used in Windows-based, web-based, enterprise, and mobile applications.
3	Goto	C++ supports the <u>goto</u> statement.	Java doesn't support the goto statement.
4	Multiple inheritance	C++ supports multiple inheritance.	Java doesn't support multiple inheritance through class. It can be achieved by using <u>interfaces in java</u> .
5	Operator Overloading	C++ supports <u>operator overloading</u> .	Java doesn't support operator overloading.
6	Pointers	C++ supports <u>pointers</u> . You can write a pointer program in C++.	Java supports pointer internally. However, you can't write the pointer program in java. It means java has restricted pointer support in java.
7	Call by Value and Call by reference	C++ supports both call by value and call by reference.	Java supports call by value only. There is no call by reference in java.
8	Structure and Union	C++ supports structures and unions.	Java doesn't support structures and unions.
9	Hardware	C++ is nearer to hardware.	Java is not so interactive with hardware.
10	Virtual Keyword	C++ supports virtual keyword so that we can decide whether or not to override a function.	Java has no virtual keyword. We can override all non-static methods by default. In other words, non-static methods are virtual by default.

General Structure of JAVA program

A typical structure of a Java program contains the following elements:

Optional section	
1.	Document section
2.	Package statement
3.	Import statement
4.	Interface statement
Compulsory section	
5.	Class definition
6.	Main method class

A java program may contain one or more section

Main method and class is essential section(5 and 6 compulsory) and other are optional

1. Document section:

Document section contain a set of comments lines giving the name of the program, the author and others details which the programmer would like to refer to at a later stage.

2. Package Statements:

The first statement allowed in JAVA is a Package statement. It declare a package name and informs the compiler that classes define here belong to these package.

3. Import statement:

This is similar to the #include statement in c.

Syntax: import packagename.classname;

Example: import java.io.*;

The above statement instruct the interpreter to load the IO class contain in the package java.

4. Interface statement:

An interface is like class but include a group of method declaration. This is also an optional section and it is used also when we wish to implement the multiple inheritance features in program.

[Compulsory Section]

5. Class definition:

A java program may contain multiple class definition. Classes are primary and essential elements of java.

6. Main method class:

Since every java stand-alone program required a main(). As it is starting point this is the essential part of java program. All java application begin execution by calling main(). A simple java program contain only this part.

Creating Your First Application

To create this program:

- **Create a source file**

A source file contains code, written in the Java programming language, that you and other programmers can understand. You can use any text editor to create and edit source files.

- **Compile the source file into HelloWorld .class file**

The Java programming language compiler (javac) takes your source file and translates its text into instructions that the Java virtual machine can understand. The instructions contained within this file are known as bytecodes.

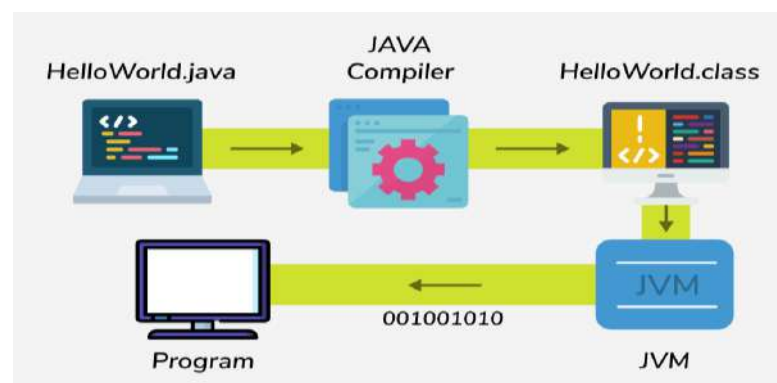
- **Run the program**

The Java application launcher tool (java) uses the Java virtual machine to run your application.

Create a Source File: To create a source file:

First, start your editor. You can launch the Notepad++ editor from the Start menu by selecting Programs > Accessories > Notepad++. In a new document, type in the following code:

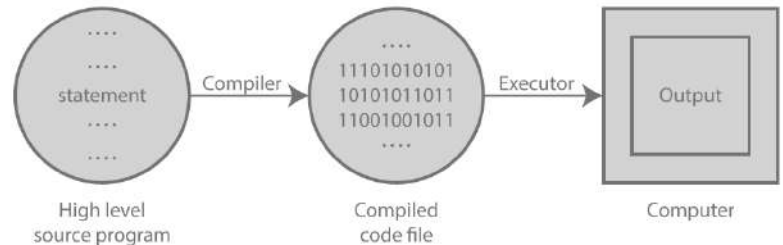
```
class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello World!"); // Display the string.  
    }  
}
```



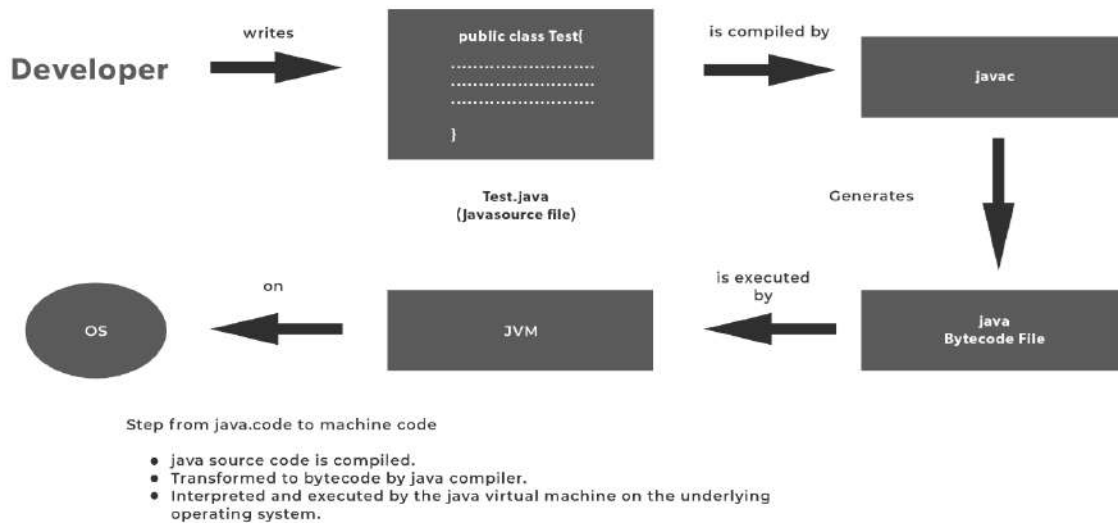
1.3 Java Compiler, Java Interpreter

1.3.1 Java Compiler

A compiler in Java translates the entire source code into a machine-code file or any intermediate code, and that file is then executed. It is platform-independent. A bytecode is basically an intermediate code generated by the compiler after the compilation of its source code. Java compiler can be operated by using the “Javac.exe” command from the command prompt. Some of the compiler options are given below:

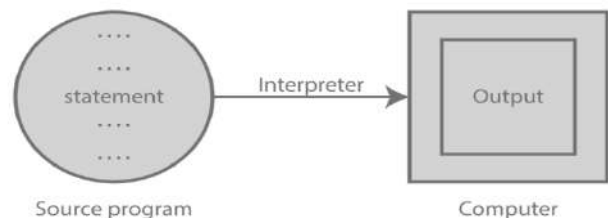


Java Compiler Step By Step

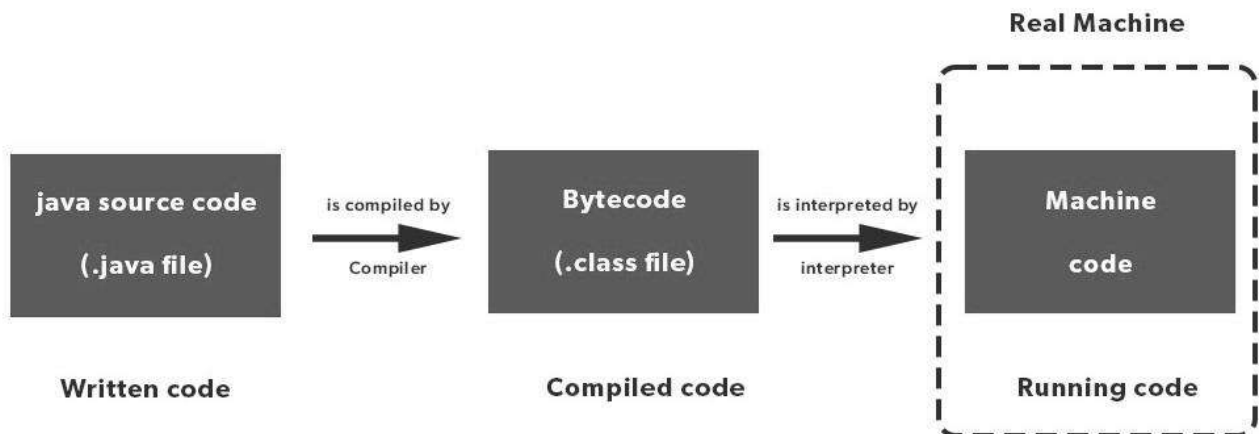


1.3.2 Java Interpreter

Similarly, an Interpreter is a computer program that converts the high-level program statement into Assembly-level language. It converts the code into machine code when the program is run. Some of the interpreter options are given below:



Java Interpreter Step By Step



1.4 Identifier, Literals, Operators, Variables, Keywords, Data Types

Basic Syntax of Java

1. **Java File Extension:**
 - A Java source file is saved with a .java extension.
2. **Java Class File:**
 - When a .java file is successfully compiled, it generates a corresponding .class file containing the bytecode.
3. **Java File Name and Class Name:**
 - The name of the Java source file must match the name of the main class it contains.
 - In the case of multiple classes in a single .java file, the class containing the main() method is considered the main class.
4. **The main Method:**
 - A Java program must include a main() method, which serves as the entry point for execution. The method is defined as:
 - `public static void main(String args[]) {}`
5. **Case Sensitivity:**
 - Java is a case-sensitive programming language. For example, Main and main are treated as different identifiers in Java.

Comments in Java

Comments are non-executable lines to explain program and author details which make program understandable by programmer though it is not created by developer:

In java comments can be define by following way.

1. **Single line comments**
2. **Multiline comments**

1. Single line comments

Single line comment specify by // (double forward slash)

For Example:

Method 1: specify comment in separate line

```
// This is a single line comment
```

```
System.out.println(" Java Programming Language ");
```

Method 2: Comment in same line

```
System.out.println(" Java Programming Language "); // This is a single line comment
```

2. Multiline comments

Multiline comments specify by `/*` and `*/` any text between `/*` and `*/` will be consider as non-executable line.

For Example:

```
/* This program is illustration of multiline comments
```

```
To print greetings */
```

```
System.out.println("Welcome to Java Programming Language");
```

1.4.1 JAVA Identifier

An identifier refers to any valid name used to represent classes, variables, methods, or functions in Java. It is the name we assign to these program elements.

Rules for Defining Valid Identifiers in Java:

1. Identifiers must begin with a letter (A-Z or a-z), a currency symbol (\$), or an underscore (_).
2. After the first character, identifiers can include any combination of letters, digits (0-9), currency symbols, or underscores.
3. Reserved keywords in Java cannot be used as identifiers.
4. Identifiers are case-sensitive. For example, Name and name are treated as different identifiers.

Examples:Valid Identifiers:

age, \$charges, _point, __1_value

Invalid Identifiers:

1a (starts with a digit), -ve (contains a special character not allowed).

1.4.2 Java Literals

Any constant value which can be assigned to the variable is called as literal/constant

1. Integer Literals

- Most Commonly used type in typical program is any whole number value is an integer literal.
- **For Example:** 1, 2, 3 and 25
- Note: byte, short, long literals value is also type as same Integer literals

2. Floating Point Literals

- Floating point number represents decimal values with fractional components.
- **For Example:** 6.023E23, 3.1415

3. Boolean Literals

- There are only two logical values that a Boolean value can have **true** or **false**

4. Character Literals

- Characters in JAVA are indicates into the Unicode character set. Character literal is represent inside a pair of single quote (' ')
- **For Example:** 'x','y'

5. String Literals

- String literals in JAVA are specified like they are in most other languages by enclosing a sequence of character between a pair of double quotes (" ")
- **For Example:** "Hello World" , "SYBCA division 3 and 4"

6. The Null Literal

- The null type has one value, the null reference, represented by the literal null, which is formed from ASCII characters. A null literal is always of the null type.
NullLiteral:
- **For Example:** null

1.4.3 Operators in Java

Operators in Java are the symbols used for performing specific operations in Java. Operators are the building blocks of Java expressions, allowing you to perform calculations, comparisons, and more.

Types of Operators in Java

There are multiple types of operators in Java all are mentioned below:

1. Arithmetic Operators
2. Unary Operators
3. Assignment Operator
4. Relational Operators
5. Logical Operators
6. Ternary Operator
7. Bitwise Operators
8. Shift Operators

1. Arithmetic Operators

These operators are used to perform basic mathematical operations.

Operator	Description	Example	Result
+	Addition	int sum = 5 + 3;	8
-	Subtraction	int diff = 10 - 4;	6
*	Multiplication	int product = 7 * 2;	14
/	Division	int quotient = 20 / 4;	5
%	Modulus (remainder)	int remainder = 17 % 3;	2

2. Unary Operators

Operate on a single operand.

Operator	Description	Example	Result
+	Unary plus	int num = +5;	5
-	Unary minus	int num = -5;	-5

Operator	Description	Example	Result
++	Increment by 1 (postfix)	int a = 5; a++;	6
--	Decrement by 1 (prefix)	int b = 5; --b;	4
!	Logical NOT	boolean flag = false;	true

3. Assignment Operators

Used to assign values to variables.

Operator	Description	Example	Result
=	Assign value	int x = 10; x = 10	
+=	Add and assign	x += 5;	x = 15
-=	Subtract and assign	x -= 3;	x = 12
*=	Multiply and assign	x *= 2;	x = 24
/=	Divide and assign	x /= 4;	x = 6
%=	Modulus and assign	x %= 2;	x = 0

4. Relational Operators

Compare two values and return a boolean result.

Operator	Description	Example	Result
==	Equal to	5 == 5	true
!=	Not equal to	5 != 3	true
>	Greater than	7 > 5	true
<	Less than	4 < 6	true
>=	Greater or equal to	5 >= 5	true

Operator	Description	Example	Result
<=	Less or equal to	4 <= 6	true

5. Logical Operators

Used to perform logical operations on boolean expressions.

Operator	Description	Example	Result
&&	Logical AND	(5 > 3) && (6 > 4)	true
	Logical OR	x < 5 x < 4	true
!	Logical NOT	!(5 > 3)	false

6. Ternary Operator

A shorthand for if-else statements.

Operator	Description	Example	Result
?:	Ternary conditional	int max = (a > b) ? a : b;	max = a (if a > b)

7. Bitwise Operators

Perform bit-level operations.

Operator	Symbol	Uses
Bitwise AND	&	op1 & op2
Bitwise NOT	^	op1 ^ op2
Bitwise OR		op1 op2
Bitwise Compliment	~	~ op
Bitwise left shift	<<	op1 << op2
Bitwise right shift	>>	op1 >> op2

Example :

```
int p=9,q=10; //BINARY 9=1001 10=1010

//bitwiseAnd
System.out.println(p&q); //(9 = 1001 & 10 = 1010 ANS= 8 = 1000)

//bitwiseOR
System.out.println(p|q); //(9=1001 | 10=1010 ANS= 11=1011)
System.out.println(Integer.toBinaryString(p|q)); //(9=1001 | 10=1010 ANS= 11=1011)

//bitwiseNot
System.out.println(p^q); //(9 = 1001 ^ 10 = 1010 ANS= -3 = 0011)

//bitwise complement
System.out.println(~p); // 9=1001 ~9=0110
//System.out.println(Integer.toBinaryString(~p));

//left shift a<<b = a*2 power of b
System.out.println(p<<1); // 9=1001 << 1 bitwise left shift 9*2(1) = 18 = 10010

//left shift a<<b = a*2 power of b
System.out.println(p<<2); // 9=1001 << 2 bitwise left shift 9*2(2) = 36 = 100100

//RIGHT shift a>>b = a/2 power of b
System.out.println(p>>1); // 9=1001 >> 1 bitwise right shift 9/2(1) = 4 = 100

//RIGHT shift a>>b = a/2 power of b
System.out.println(p>>2); // 9=1001 >> 2 bitwise right shift 9/2(2) = 2 = 10
```

8 Bitwise Shift Operators

Shift operators in Java are used to manipulate the bits of integer types (byte, short, int, and long) by shifting their binary representations to the left or right. These operators are often employed for low-level programming, optimizing calculations, and bitwise operations.

Types of Shift Operators

There are three types of shift operators in Java:

1. Left Shift (<<)
2. Right Shift (>>)
3. Unsigned Right Shift (>>>)

Operator	Description	Example	Result
<<	Left shift	5 << 2	20
>>	Right shift	20 >> 2	5
>>>	Unsigned right shift	-20 >>> 2	Positive value

1. Left Shift Operator (<<)

The left shift operator shifts all the bits in a binary number to the left by a specified number of positions. The vacant rightmost bits are filled with 0.

Example:

```
int num = 5; // Binary: 00000000 00000000 00000000 00000101
int result = num << 2; // Binary: 00000000 00000000 00000000 00010100
System.out.println(result); // Output: 20
```

2. Right Shift Operator (>>)

The right shift operator shifts all the bits in a binary number to the right by a specified number of positions. The leftmost bits are filled with the **sign bit** (0 for positive numbers, 1 for negative numbers).

Example:

```
int num = 20; // Binary: 00000000 00000000 00000000 00010100
int result = num >> 2; // Binary: 00000000 00000000 00000000 00000101
System.out.println(result); // Output: 5
```

Example For Negative Numbers:

```
int num = -20; // Binary: 11111111 11111111 11111111 11101100
int result = num >> 2; // Binary: 11111111 11111111 11111111 11111011
System.out.println(result); // Output: -5
```

3. Unsigned Right Shift Operator (>>>)

The unsigned right shift operator shifts all the bits to the right by a specified number of positions. Unlike the right shift operator (>>), it does **not preserve the sign bit** and fills the leftmost bits with 0.

Example:

```
int num = 20; // Binary: 00000000 00000000 00000000 00010100
int result = num >>> 2; // Binary: 00000000 00000000 00000000 00000101
System.out.println(result); // Output: 5
```

Example For Negative Numbers:

```
int num = -20; // Binary: 11111111 11111111 11111111 11101100
int result = num >>> 2; // Binary: 00111111 11111111 11111111 11111011
System.out.println(result); // Output: 1073741819
```

1.4.5 JAVA Keywords

Java has a set of keywords that are reserved words that cannot be used as variables, methods, classes, or any other identifiers. Following are the keywords in java:

1. abstract	13. double	25. int	37. strictfp
2. assert	14. else	26. interface	38. super
3. boolean	15. enum	27. long	39. switch
4. break	16. extends	28. native	40. synchronized
5. byte	17. final	29. new	41. this
6. case	18. finally	30. package	42. throw
7. catch	19. float	31. private	43. throws
8. char	20. for	32. protected	44. transient
9. class	21. if	33. public	45. try
10. continue	22. implements	34. return	46. void
11. default	23. import	35. short	47. volatile
12. do	24. instanceof	36. static	48. while

1.4.6 JAVA Data Types

In Java, data types define the type of data a variable can store. They are fundamental to writing efficient and type-safe code. Java is a statically-typed language, meaning variables must be declared with a specific data type before use.

Java data types are broadly classified into two categories:

- 1. Primitive Data Types:**
Primitive types optimize memory and performance for simple tasks.
- 2. Non-Primitive Data Types:**
Non-primitive types enable complex data modeling, such as storing objects, arrays, or user-defined data structures.

1. Primitive Data Types

These are the most basic data types built into the Java language. Java has 8 primitive data types, each optimized for specific purposes.

Data Type	Size	Default Value	Range	Description
byte	1 byte (8 bits)	0	-128 to 127	Stores small integers. Useful for saving memory.
short	2 bytes (16 bits)	0	-32,768 to 32,767	Stores medium-range integers.
int	4 bytes (32 bits)	0	-2^{31} to $2^{31} - 1$	Default type for integers.
long	8 bytes (64 bits)	0L	-2^{63} to $2^{63} - 1$	Stores large integers. Append L to values.
float	4 bytes (32 bits)	0.0f	$\pm 3.40282347\text{E}+38\text{F}$	Stores fractional numbers, up to 7 decimal digits. Append f.
double	8 bytes (64 bits)	0.0d	$\pm 1.79769313486231570\text{E}+308$	Stores fractional numbers, up to 16 decimal digits. Append d.
char	2 bytes (16 bits)	'\u0000'	0 to 65,535 (Unicode values)	Stores a single character or Unicode value. Use single quotes.
boolean	1 bit	false	true or false	Stores logical values.

Examples of Primitive Data Types

```
byte smallNumber = 120;
short mediumNumber = 32000;
int age = 25;
long largeNumber = 123456789L;
float percentage = 75.5f;
double pi = 3.14159265359;
char grade = 'A';
boolean isJavaFun = true;
```

2. Non-Primitive Data Types

Non-primitive data types (also known as reference types) are more complex and can store multiple values or objects. These types are derived from Java classes or interfaces.

Type	Description
String	Used to store sequences of characters.
Array	A collection of elements of the same type.
Class	Represents user-defined or built-in objects.
Interface	Defines methods that must be implemented by classes.

Examples of Non-Primitive Data Types

```
String name = "John";
int[] numbers = {1, 2, 3, 4, 5};
Student student = new Student(); // `Student` is a user-defined class.
```

Key Differences Between Primitive and Non-Primitive Data Types

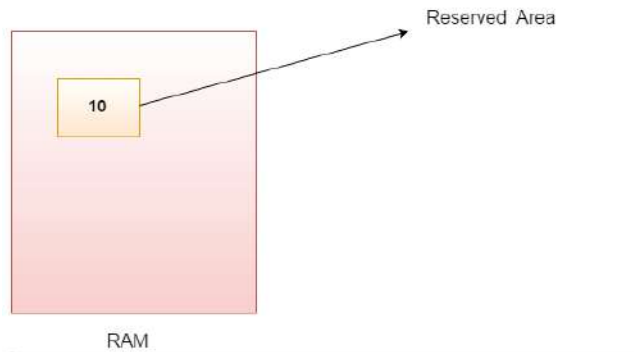
Aspect	Primitive Types	Non-Primitive Types
Definition	Built into Java.	Created by the programmer.
Memory	Stored in stack memory.	Stored in heap memory.
Default Values	Have specific default values (e.g., 0).	Null is the default value.
Operations	Can only perform simple operations.	Can have methods and behaviors.

JAVA Variables

A variable is a container which holds the value while the Java program is executed. A variable is assigned with a data type.

Variable is a name of memory location. There are three types of variables in java: local, instance and static.

Variable: A variable is the name of a reserved area allocated in memory. In other words, it is a name of the memory location. It is a combination of "vary + able" which means its value can be changed.



```
int data=50;//Here data is variable
```

Types of Variables

There are three types of variables in Java:

1. local variable
2. instance variable
3. static variable

1) Local Variable

A variable declared inside the body of the method is called local variable. You can use this variable only within that method and the other methods in the class aren't even aware that the variable exists.

A local variable cannot be defined with "static" keyword.

2) Instance Variable

A variable declared inside the class but outside the body of the method, is called an instance variable. It is not declared as static.

It is called an instance variable because its value is instance-specific and is not shared among instances .

3) Static variable

A variable that is declared as static is called a static variable. It cannot be local. You can create a single copy of the static variable and share it among all the instances of the class. Memory allocation for static variables happens only once when the class is loaded in the memory.

JAVA Control Structure : Branching and Looping Statements in Java

Java provides a variety of **branching** and **looping** statements to control the flow of execution in a program. These statements enable decision-making and repetitive execution of code blocks.

1.5 Branching Statements

Branching statements in Java are used for decision-making, allowing the program to execute certain sections of code based on specific conditions.

1. if Statement

The if statement executes a block of code only if the given condition is true.

Syntax <pre>if (condition) { // Code to execute if condition is true }</pre>	Example <pre>int age = 18; if (age >= 18) { System.out.println("Eligible to vote."); }</pre>
--	--

2. if-else Statement

The if-else statement provides an alternative block of code to execute if the condition is false.

Syntax: <pre>if (condition) { // Code to execute if condition is true } else { // Code to execute if condition is false }</pre>	Example: <pre>int number = 10; if (number % 2 == 0) { System.out.println("Even number"); } else { System.out.println("Odd number"); }</pre>
---	---

3. if-else if Ladder

The if-else if ladder is used to check multiple conditions sequentially.

Syntax <pre>if (condition1) { // Code if condition1 is true }</pre>	Example <pre>int marks = 75; if (marks >= 90) { }</pre>
---	--

<pre> } else if (condition2) { // Code if condition2 is true } else { // Code if none of the conditions are true } </pre>	<pre> System.out.println("Grade A"); } else if (marks >= 75) { System.out.println("Grade B"); } else if (marks >= 50) { System.out.println("Grade C"); } else { System.out.println("Fail"); } </pre>
---	--

4. switch Statement

The switch statement tests a variable for equality against a list of values (case) and executes the corresponding block.

Syntax: <pre> switch (expression) { case value1: // Code for value1 break; case value2: // Code for value2 break; default: // Code if no case matches } </pre>	Example: <pre> int day = 3; switch (day) { case 1: System.out.println("Monday"); break; case 2: System.out.println("Tuesday"); break; case 3: System.out.println("Wednesday"); break; default: System.out.println("Invalid day"); } </pre>
---	--

1.6 Looping Statements

Looping statements in Java are used to execute a block of code repeatedly as long as a condition is true.

1. for Loop

The for loop is used when the number of iterations is known.

Syntax:	Example:
<pre>for (initialization; condition; update) { // Code to execute in each iteration }</pre>	<pre>for (int i = 1; i <= 5; i++) { System.out.println("Iteration: " + i); }</pre>

2. Enhanced for Loop (For-Each Loop)

The enhanced for loop is used to iterate through arrays or collections.

Syntax:	Example:
<pre>for (type variable : array) { // Code to execute for each element }</pre>	<pre>int[] numbers = {10, 20, 30, 40}; for (int num : numbers) { System.out.println(num); }</pre>

3. while Loop

The while loop is used when the number of iterations is unknown and depends on a condition.

Syntax:	Example:
<pre>while (condition) { // Code to execute while condition is true }</pre>	<pre>int i = 1; while (i <= 5) { System.out.println("Iteration: " + i); i++; }</pre>

4. do-while Loop

The do-while loop is similar to the while loop but guarantees at least one execution of the code block.

Syntax: <pre>do { // Code to execute } while (condition);</pre>	Example: <pre>int i = 1; do { System.out.println("Iteration: " + i); i++; } while (i <= 5);</pre>
---	--

1.7 Type Casting

Type casting in Java refers to converting a variable of one data type to another. It is particularly useful when working with mixed data types or to optimize operations in Java programs. Since Java is a statically-typed language, type casting is often necessary to perform operations on variables of different types.

Casting Rules:

1. **Widening:** Always safe; no need for explicit casting.
2. **Narrowing:** Use cautiously to avoid data loss or unexpected results.
3. **Type Safety:** Always ensure the cast value is within the target type's range.
4. **Avoid Redundant Casts:** Java often detects unnecessary casting and generates warnings.

There are two primary types of type casting in Java:

1. Widening (Implicit) Type Casting
2. Narrowing (Explicit) Type Casting

1. Widening (Implicit) Type Casting

- **Definition:** Widening type casting happens automatically when converting a smaller data type to a larger data type. This is safe and does not result in data loss.
- **Data Flow:** Smaller → Larger
- **Supported Conversions:**
 - byte → short → int → long → float → double
 - char → int

Syntax: <pre>largerType variable = smallerTypeVariable;</pre>	Example: <pre>int num = 10; double result = num; // Implicit conversion from int to double System.out.println("Result: " + result); // Output: Result: 10.0</pre>
---	---

Note: Larger data types can easily accommodate the values of smaller data types without any loss of precision or data.

2. Narrowing (Explicit) Type Casting

- **Definition:** Narrowing type casting is done manually to convert a larger data type into a smaller data type. It may result in data loss or precision errors.
- **Data Flow:** Larger → Smaller
- **Supported Conversions:**
 - double → float → long → int → short → byte
 - int → char (if within the range of char)

Syntax:	Example:
<pre>smallerType variable = (smallerType) largerTypeVariable;</pre>	<pre>double value = 10.5; int result = (int) value; // Explicit conversion from double to int System.out.println("Result: " + result); // Output: Result: 10</pre>

Note: Narrowing is not safe because :

- **Data Loss:** Precision can be lost when converting types (e.g., 10.5 becomes 10).
- **Range Errors:** If the value exceeds the range of the smaller data type, it wraps around or causes unpredictable results.

Example:

<pre>int largeValue = 150; byte smallValue = (byte) largeValue; // Value exceeds byte's range (-128 to 127) System.out.println("Small Value: " + smallValue); // Output: Small Value: -106</pre>
